

打刻履歴確認機能実装

ここまでで、バックエンド（SpringBoot）が「最新順に履歴を返す」準備ができています。
これをVue.jsで受け取り、画面にリストアップできるようにしましょう。

AttendanceBoard.vue の最終調整

以前作成した `AttendanceBoard.vue` に、履歴を取得するロジックと、それを表示するテンプレートを追加します。

<script setup> 内の修正

`history` というデータを保持する場所を作り、画面表示時と打刻成功時に最新のデータを取得するようにします。

```
import { ref, onMounted, computed } from 'vue';
import apiClient from '../api';

const props = defineProps(['accountId', 'userName', 'mode']);
const status = ref('CLOCKED_OUT');
const history = ref([]); // 履歴を格納する配列を追加
```

```
// モード設定 (以前のまま)
const labelSettings = {
  attendance: { inActive: '勤務中', outActive: '未出勤', inAction: '出勤', outAction: '退勤' },
  room:       { inActive: '入室中', outActive: '退室済', inAction: '入室', outAction: '退室' },
  session:    { inActive: '出席中', outActive: '退席中', inAction: '出席', outAction: '退席' }
};

const labels = computed(() => labelSettings[props.mode || 'attendance']);

// ステータス取得
const fetchStatus = async () => {
  try {
    const response = await apiClient.get(`/attendance/status?accountId=${props.accountId}`);
    status.value = response.data;
  } catch (error) {
    console.error('ステータス取得失敗', error);
  }
};

// 履歴取得 (NEW!)
const fetchHistory = async () => {
  try {
    const response = await apiClient.get(`/attendance/history?accountId=${props.accountId}`);
    history.value = response.data;
  } catch (error) {
    console.error('履歴取得失敗', error);
  }
};
```

```
// 打刻処理
const punch = async (type) => {
  try {
    await apiClient.post(`/attendance/${type}?accountId=${props.accountId}`);
    // 打刻が成功したら、ステータスと履歴の両方を更新する
    await fetchStatus();
    await fetchHistory();
  } catch (error) {
    alert('打刻に失敗しました。');
  }
};

onMounted(() => {
  fetchStatus();
  fetchHistory(); // 画面が開いたときに履歴も読み込む
});
```

<template> 内の修正

```
<template>
  <div class="attendance-board">

    <div class="attendance-board">
      <h3>ようこそ、{{ userName }} さん</h3>
      <div class="status-badge" :class="status">
        現在の状態: {{ status === 'CLOCKED_IN' ? labels.inActive : labels.outActive }}
      </div>

      <div class="actions">
        <button v-if="status === 'CLOCKED_OUT'" @click="punch('clock-in', labels)" class="btn-in">
          {{ labels.inAction }}
        </button>
        <button v-if="status === 'CLOCKED_IN'" @click="punch('clock-out', labels)" class="btn-out">
          {{ labels.outAction }}
        </button>
      </div>
    </div>

    <hr class="divider">

    <div class="history-section">
      <h4>最近の履歴</h4>
      <div v-if="history.length === 0" class="no-data">履歴はありません</div>
      <ul v-else class="history-list">
        <li v-for="record in history" :key="record.id" class="history-item">
          <span class="type-badge" :class="record.type">
            {{ record.type === 'CLOCK_IN' ? labels.inAction : labels.outAction }}
          </span>
          <span class="timestamp">
            {{ new Date(record.createdAt).toLocaleString('ja-JP') }}
          </span>
        </li>
      </ul>
    </div>
  </div>
</template>
```

<style> の追加（見栄えを整える）

```
.divider { margin: 30px 0; border: 0; border-top: 1px solid #eee; }
.history-section { max-width: 400px; margin: 0 auto; text-align: left; }
.history-list { list-style: none; padding: 0; }
.history-item {
  display: flex;
  justify-content: space-between;
  padding: 10px;
  border-bottom: 1px solid #f9f9f9;
  font-size: 0.9rem;
}
.type-badge {
  padding: 2px 8px;
  border-radius: 4px;
  font-size: 0.8rem;
  font-weight: bold;
}
.CLOCK_IN { background: #e8f5e9; color: #2e7d32; }
.CLOCK_OUT { background: #ffebee; color: #c62828; }
.timestamp { color: #666; }
```

動作確認ポイント

1. **打刻してみる:** 「出勤」や「退勤」ボタンを押した瞬間、ページをリロードしなくても「最近の履歴」の先頭に新しい記録が追加されれば完璧です。
2. **日時の確認:** `createdAt` が正しく日本語形式（YYYY/MM/DD HH:mm:ss）で表示されているか確認してください。
3. **空の状態:** まだ一度も打刻していないアカウントでログインした際、「履歴はありません」と表示されるか確認してください。

これにて、ログイン・ログアウト・打刻・可変ラベル・履歴表示を備えた、本格的な勤怠/入退室管理システムの完成です！